

一. 问题分析

1. 分析并确定要处理的对象是什么

要处理的对象是每组数据中输入的 n (珊瑚礁列数)与 x (最多可用水量)与数组 $a[0 \cdots n-1]$ (每列珊瑚礁的初始高度构成的数组)

2. 分析并确定要实现的功能是什么

处理目标: 决定一个整数水位高度 h , 在两侧建高度为 h 的“水墙”, 向中间注水, 使得注入总量不超过 x , 并在所有满足该约束的 h 中取 最大值。

最终确定每个水族箱满足条件的最大高度 h 并输出

3. 分析并确定处理后的结果如何显示

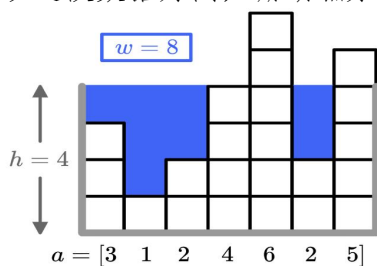
每个测试用例的结果应单独占一行, 直接输出一个正整数 (最大可行的 h)

4. 用题目中样例, 详细给出样例求解过程

7 9

3 1 2 4 6 2 5

以这段数据为例, 珊瑚礁分布如图所示



(1) 将珊瑚礁高度排序, 构成递增序列, 方便后续进行二分查找

$a[i]$ 为: 1 2 2 3 4 5 6

(2) 计算前缀和, 方便后续代入公式计算需要的总水量

前缀和 $pre = [0, 1, 3, 5, 8, 12, 17, 23]$, 其中 $pre[i]$ 为小标 i 区间之和

(3) 确定二分区间: 理论上水位最高不可能超过 $(pre(a) + x) / n \approx (23 + 9) / 7 \approx 4.57$, 取整后+1为6, 故取上界 $high = 6$ 下界 $low = 1$ 。

(4) 二分查找

迭代1:

• $mid = (1 + 6) / 2 = 3$

• 定位多少列 < 3 : $k = lower_bound(a, 3) - a.begin() = 3$ (对应 $a[0..2] = [1, 2, 2]$)

• 注水量 $need = 3 * 3 - pre[3] = 9 - 5 = 4$, $4 \leq 9$,

合法 $\rightarrow ans = 3$, $low = mid + 1 = 4$

迭代2:

• $mid = (4 + 6) / 2 = 5$

• $k = lower_bound(a, 5) = 5$ (前5列 < 5)

• $need = 5 * 5 - pre[5] = 25 - 12 = 13$, $13 > 9$,

不行 $\rightarrow high = mid - 1 = 4$

迭代3:

• $mid = (4 + 4) / 2 = 4$

• $k = lower_bound(a, 4) = 4$

• $need = 4 * 4 - pre[4] = 16 - 8 = 8$, $8 \leq 9$,

合法 $\rightarrow ans = 4$, $low = 5$

循环结束 ($low = 5 > high = 4$), 最终 $ans = 4$ 即为该组答案。

二. 数据结构与算法设计

1. 抽象数据类型：（本题主要使用二分查找算法，未使用明显的数据结构知识，此处将动态数组vector视为抽象数据类型）

ADT: vector

数据对象：珊瑚高度的序列及前缀和序列

基本操作：

(1) 排序: `sort(a.begin(), a.end())`（对序列进行升序排列）

(2) 有序查找: `lower_bound(a.begin(), a.end(), mid)`（二分下界查找）

2. 物理数据对象设计

(1) 高度数组 `vector<long long> a(n)`: 动态分配, 使用STL容器存储每一列珊瑚的高度

(2) 前缀和数组 `vector<long long> pre(n+1)`: 与高度数组等长+1, 存储前缀和用于快速计算对前n列的总高度之和

3. 算法思想的设计

核心思想: 使用二分查找与前缀和的组合解决本题: 在得到前缀和基础上, 使用二分查找更新mid值, 对当前高度的用水量与给定水量比较并动态更新low值或high值进而更新mid值, 直到退出循环, 找到最大h值

- 首先进行排序, 因为二分查找要求有序序列: 先对数组 `a[i]` 从小到大排序, 这样在判断给定 `h` 时, 可以快速定位哪些列需要补水。

`sort(a.begin(), a.end());`

- 然后计算前缀和构造排序后数组的前缀和 $pre[k] = \sum_{i=0}^{k-1} a_i$ 用来快速计算前k列的总高度之和。

`vector<long long> pre(n+1, 0);`

```
for(int i = 0; i < n; i++){
    pre[i+1] = pre[i] + a[i];
}
```

- 再计算low和high进行二分查找。low的初始值定为low=1; high的最大值计为 $high = (x + total) / n + 1$ 这是因为将整个池子补全为长方形, ($total+x$) 为总单位数=高度×列数。(total为珊瑚总单位数, x为最大注水量)

- 然后开始二分查找最大h, 使得 $\sum \max(0, h-a[i]) \leq x$, 计算每一个mid对应的最大列高 $a[i] < mid$, 计算此处需要的水量与x的大小关系, 再进行动态更新mid的值。直到low>high退出循环

`while(low <= high) {`

```
    long long mid = low + (high - low) / 2;
    int k = lower_bound(a.begin(), a.end(), mid) - a.begin();
    long long need = mid * k - pre[k];
    if(need <= x) {
        ans = mid;
        low = mid + 1;
    } else {
        high = mid - 1;
    }
}
```

`cout << ans << "\n";`

`}`

4. 关键步骤的算法步骤

本题关键步骤为二分查找，下面解释二分查找的算法执行过程：

二分查找预备工作得到了 $a[i]$ 的升序排列， $pre[n]$ 前缀和与 h 可能的最大值。

第一步：计算 mid 值为 $low + (high - low) / 2$ ，这是二分查找寻找中间值的常见操作。

第二步：计算有多少个 $a[i]$ 小于 mid 的值，进而在排好序的 a 中，找到第一个不小于 mid 的元素位置。 k 就是这个位置的下标，也就是序列中所有小于 mid 的元素个数。

第三步：计算所需水量：

推导公式： $need = (mid - a[1] + mid - a[2] + \dots + mid - a[k]) = mid * k - pre[k]$

而之前已经得到了 $pre[k]$ ，即可计算出需要的水量

第三步：更新

如果 $need$ 小于 x ，说明 mid 合法，可以尝试更高的高度；更新 $low = mid + 1$ ；

如果 $need$ 大于 x ，说明 mid 高了，需要调低高度， $high = mid - 1$

第四步：执行循环，直到 $low > high$ ，输出 ans ；

三. 算法性能分析

1. 时间复杂度

排序`sort(a.begin(), a.end());`；

需要 $O(n \log n)$ 时间

构造前缀和

```
for (int i = 0; i < n; i++)
```

```
pre[i+1] = pre[i] + a[i];
```

需要 $O(n)$ 时间

二分查找

迭代次数：设二分区间宽度约为 H ，故迭代次数为 $O(\log H)$ 。

每次迭代代价：`lower_bound` 在已排序数组上二分查找，耗时 $O(\log N)$

共 $O(\log H * \log N)$

综上：时间复杂度为 $O(n \log n + n * (\log H) (\log N)) = O(n \log n + (\log H) (\log N))$

因为 $n \gg \log H$ 所以主项为 $O(n \log n)$

故时间复杂度为 $O(n \log n)$

2. 空间复杂度 $O(N)$

输入存储

`vector<long long> a(n)`：存储高度，空间 $O(n)$

`vector<long long> pre(n+1)`：存储前缀和，空间 $O(n)$

辅助变量

常数个 `int/long long (n, x, low, high, mid, ans, k, total)` 空间 $O(1)$

故空间复杂度为 $O(n)$

